CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

OBJECT ORIENTED PROGRAMMING (OOPs)
CASE STUDY

Library Management System

A Case Study on the Design, Implementation and Evaluation of a C++ Mini Project Using Object Oriented Programming Concepts

Submitted By	
Name	**Hardeep Singh Sodhi**
Class / Section	**25BCD-2(A)**
UID	**25bcd10215**

1. Executive Summary

This case study examines the design, implementation and evaluation of a console-based Library Management System (LMS) built in C++ as an Object Oriented Programming (OOP) mini project. The system replaces a manual register-and-card workflow with a menu-driven program that can add books, register two kinds of members (Students and Faculty), issue and return books, calculate late fines automatically, and search the catalogue by title.

The case is interesting less because of the library domain and more because of what the domain teaches. A library has clear real-world entities, clear rules and clear failure modes, so it is an ideal setting for demonstrating how classes, abstraction, inheritance, polymorphism, composition and exception handling work together. The central design decision, an abstract `Member` class with pure virtual functions that `Student` and `Faculty` override, allowed all member-specific rules to be kept out of the `Library` class entirely.

The finished program passed all 18 black-box test cases. This study goes beyond restating those results. It explains why the design was chosen, how the main workflows operate, where the design is strong, and where it carries risks that would matter in a production setting. It closes with concrete recommendations for evolving the project.

Key facts at a glance

Item	Detail
Language and standard	C++ (C++11)
Interface	Console, numbered menu
Core classes	`Member` (abstract), `Student`, `Faculty`, `Book`, `Library`
OOP concepts demonstrated	Encapsulation, abstraction, inheritance, polymorphism, composition, constructors/destructors, exception handling
Storage	In memory only (`vector<Book>`, `vector<Member*>`)
Test result	18 of 18 test cases passed

2. Background and Context

2.1 How a small library works

A small academic library performs a handful of repeating activities: it maintains a list of books, registers the people who may borrow them, lends books for a fixed period, takes them back, and charges a fine when a book is kept too long. Each activity is simple on its own. The difficulty is that they all depend on each other. Whether a book can be issued depends on whether it exists, whether it is already out, and how many books the borrower already holds. How much fine to charge depends on who the borrower is and how long the book was kept.

2.2 The manual process and its weaknesses

When these tasks are done with paper registers, several problems appear repeatedly:

- Finding a book or checking its availability means scanning pages.
- The librarian must remember or look up each borrower's limit.
- Fines are computed by hand and can be wrong or applied inconsistently between people.
- Records can be lost, damaged or duplicated, for example two books entered under the same number.

These are exactly the kinds of problems software handles well, because they are rule-based, repetitive and error-prone for humans.

2.3 Business rules the system must enforce

The project encodes two borrower categories with different policies:

Rule	Student	Faculty
Maximum books at a time	3	5
Loan period	14 days	30 days
Fine per late day	Rs. 2	Rs. 1

Fine = (days kept - loan period) × fine per day, charged only when the result is positive. The two categories share the same formula and differ only in numbers. That observation is the key to the design: the *structure* of the rules is common, and the *values* vary by member type.

3. Problem Statement and Objectives

3.1 Problem statement

How can a small library's book and member records, and its issue and return process, be automated in a way that applies different rules to different member types, reports every invalid operation clearly, and remains easy to extend when a new member type is introduced?

3.2 Objectives

The project set out to:

1. Apply the core OOP concepts to a realistic problem rather than to isolated textbook examples.
2. Store books and members using classes and the STL `vector` container.
3. Automate issuing, returning and fine calculation.
4. Use abstract classes and virtual functions so that different member types follow different rules through one common interface.
5. Manage dynamic memory safely, with proper destructors.
6. Use exception handling to report duplicate IDs, missing records and exceeded limits without crashing.
7. Produce a modular, readable program that can be extended.

3.3 Scope

In scope are the core functions of a small library: books and members with unique IDs, issuing with validation, returning with fine calculation, title search and record display. Out of scope for this version are permanent storage, a graphical interface, online access and real calendar dates. The number of days a book was kept is typed in by the librarian rather than computed from dates.

4. Analysis and Approach

4.1 Finding the classes

A standard technique in object oriented analysis is to read the problem description and underline the nouns. Doing so yields *book*, *member*, *student*, *faculty* and *library*. Each of these became a class. The verbs (issue, return, search, display) became member functions, placed on whichever class owns the relevant data.

Noun in the problem	Class	Responsibility
Member	`Member` (abstract)	Common identity and issue count; declares the rules every member type must define
Student	`Student`	Supplies the student rules
Faculty	`Faculty`	Supplies the faculty rules
Book	`Book`	Holds book data and its availability state
Library	`Library`	Owns all records and coordinates the workflows

4.2 Requirements

The analysis produced seven functional requirements: add a book, register a member, issue a book within limit, return a book with fine, search by title keyword, display books and members, and show a proper error message for every invalid operation. The non-functional requirements were usability (a simple numbered menu), reliability (no crash on wrong IDs or duplicates), maintainability (one responsibility per class), extensibility (new member types by deriving from `Member`) and performance (instant for the small data size).

The single actor is the Librarian. Six use cases map to the menu: Add Book, Register Member, Issue Book, Return Book, Search Book and View Records.

4.3 Alternatives considered

It is worth asking what the project could have looked like without OOP. A procedural version would likely store members in a structure with a `type` field and use `if (type == STUDENT) ... else ...` branches wherever a rule is needed: when checking the limit, when stating the loan period, and when computing the fine. This works for two types, but every new type forces edits in every one of those branches, and a missed branch is a silent bug.

A second alternative is a single `Member` class with `maxBooks`, `loanDays` and `finePerDay` stored as ordinary data fields, set by the constructor according to a type argument. This is simpler and, honestly, adequate for a pure data difference. The virtual-function approach is justified mainly when behaviour, not just numbers, may differ between types in future (for example, a research scholar who is allowed to renew, or a guest who pays a deposit). The project chose polymorphism, which suits both the learning objective and the extensibility goal, and this study treats the choice as a deliberate trade-off rather than the only possible answer.

5. System Design

5.1 Class structure

The design has one inheritance hierarchy and one owning class:

- `Member` is abstract. It holds `id`, `name` and `issuedCount` as protected data and declares four pure virtual functions: `getType()`, `maxBooks()`, `loanDays()` and `finePerDay()`.
- `Student` and `Faculty` derive publicly from `Member` and override only those four functions plus a constructor.
- `Book` stores `id`, `title`, `author`, `available` and `issuedTo` as private data.
- `Library` privately holds a `vector<Book>` and a `vector<Member*>` and exposes the operations the menu needs.

The relationship between the classes can be summarised as follows. `Student` *is a* `Member` and `Faculty` *is a* `Member` (inheritance). The `Library` *has* books and members (composition). The `Library` *uses*

members only through `Member*` pointers (polymorphism).

5.2 Encapsulation in `Book`

All five data members of `Book` are private. The only way to change a book's state is through `issue(memberId)` and `giveBack()`, and each of these updates both `available` and `issuedTo` together. This matters because those two fields must always agree: a book that is unavailable must record who holds it, and an available book must record `-1`. By making it impossible to set one without the other, the class guarantees its own consistency. This is the practical value of encapsulation: not secrecy, but protection of an invariant.

5.3 Abstraction and polymorphism in `Member`

`Member` declares *what* every member must be able to answer (how many books, how many days, what fine) without saying *how*. Because the class contains pure virtual functions, it cannot be instantiated. A librarian cannot register a generic "member"; they must choose a Student or a Faculty. This mirrors the real rule of the library.

The payoff appears inside `Library`. When `issueBook` checks `m->canIssue()`, or `returnBook` calls `m->finePerDay()`, the `Library` neither knows nor cares which kind of member it holds. The correct overridden version runs at run time. The `Library` code contains no type tests at all.

One small but good detail: `canIssue()` is defined once in the base class as `issuedCount < maxBooks()`. Because `maxBooks()` is virtual, the same shared function gives a limit of 3 for a Student and 5 for a Faculty. This is an example of the template-method idea: common logic in the base class, variable parts supplied by the derived classes.

5.4 Memory ownership

Members are created with `new` in `main()` and handed to `Library::addMember`, which takes ownership. The `Library` destructor deletes every stored member. Two details make this safe. First, `Member` has a **virtual destructor**, so deleting a `Student` through a `Member*` runs the derived destructor correctly. Second, `addMember` deletes the pointer itself before throwing if the ID is a duplicate, so the rejected object does not leak. This second detail is easy to miss and shows careful thought about the exception path.

5.5 Error handling strategy

The `Library` functions do not print error text and do not return error codes. They `throw runtime_error` with a clear message, and the `main()` menu loop catches `exception&` and prints the message prefixed with "Error: ". This separates *detecting* a problem (deep in the logic) from *reporting* it (at the user interface), and it guarantees that one bad operation does not end the session.

6. Implementation Walkthrough

6.1 The menu loop

`main()` creates one `Library` object and runs a `do-while` loop that prints the menu, reads a choice, and uses a `switch` to dispatch. Choices 2 and 3 share one code path that differs only in whether it constructs a `Student` or a `Faculty`. The whole `switch` sits inside a `try` block, so every operation is protected by the same handler. The loop ends when the user enters 0.

6.2 Workflow: issuing a book

The issue workflow is a sequence of guard checks followed by a state change. In order, `issueBook` verifies that:

1. the book exists;
2. the member exists;

3 the book is available;

4 the member has not reached their limit.

The first failing check throws, and nothing is changed. Only when all four pass does the function call `b->issue(memberId)` and `m->incrementIssued()`, then print a confirmation that includes the member's loan period. Because all checks come before any modification, the operation is effectively atomic: it either fully succeeds or leaves the data untouched. This is a sound pattern for any transaction-like operation.

6.3 Workflow: returning a book

The return workflow checks that the book exists and is currently issued, looks up the member via `b->getIssuedTo()`, then computes `late = daysKept - m->loanDays()`. If `late` is positive, the fine is `late × m->finePerDay()`; otherwise it is zero. Finally it marks the book available, decrements the member's count and prints the result.

Notice that the librarian never enters *who* is returning the book. The book remembers its borrower through `issuedTo`, and the system recovers the member from that. This reduces input and removes the chance of crediting the return to the wrong person.

6.4 Workflow: searching

`searchByTitle` converts both the keyword and each title to lower case using `std::transform` with `::tolower`, then uses `string::find` to test for a substring. The result is a case-insensitive partial match: typing `computer` finds "Computer Networks", and typing `Object` finds "Object Oriented Programming". If nothing matches, it prints "No matching book found."

6.5 Output formatting

Both `display()` functions use `setw` and `left` from `<iomanip>` to produce aligned tables. The member table shows issued books as a ratio such as `3/3`, which communicates both current usage and the limit in one glance, using `maxBooks()` polymorphically to fill in the denominator.

7. Results

7.1 Session walk-through

Two recorded sessions exercised the system.

In **Session 1**, three books and two members (a Student, Hardeep Singh Sodhi, and a Faculty member, Dr. Anil Sharma) were added. Book 101 was issued to the student and then correctly refused when the faculty member tried to take it ("Book is already issued!"). A search for `Object` found the correct book. Book 101 was returned after 20 days, producing "Late by 6 day(s). Fine = Rs. 12", which matches the rule (20 - 14 = 6 late days × Rs. 2). A second return of the same book was refused, and issuing a non-existent book 999 was refused.

In **Session 2**, the focus was on boundaries. A duplicate book ID and a duplicate member ID were both rejected. A student was issued three books and the fourth was refused with "Issue limit reached for this member!", with the member table showing `3/3`. One return at exactly 14 days gave no fine, and another at 16 days gave Rs. 4. A keyword search with no match, an unknown member (77) and an invalid menu choice (9) were each handled cleanly.

7.2 Test coverage

The 18 test cases can be grouped by what they prove:

Group	Test cases	What it demonstrates
Record creation	1, 3, 4	Books and both member types can be added

Group	Test cases	What it demonstrates
Duplicate detection	2, 5	Unique-ID rules are enforced
Issue validation	6, 7, 8, 9, 10	Existence, availability and limit checks all work
Return and fine	11, 12, 13, 14	On-time, late and invalid returns behave correctly
Search	15, 16	Partial match and no-match
Menu	17, 18	Invalid choice and clean exit

The fine cases deserve a note. Cases 11, 12 and 13 test a boundary (exactly on the limit gives no fine) and two different late amounts, which is the right way to test a threshold rule. The fine table in the report adds a Faculty example (40 days kept, 10 late days, Rs. 10) and a Faculty on-time case, which would be worth adding as formal test cases because the report's 18 cases exercise the late-fine path only for Students.

7.3 Assessment against objectives

Objective	Outcome
Apply core OOP concepts	Achieved; each concept has an identifiable home in the code
Store records with STL `vector`	Achieved
Automate issue, return and fine	Achieved and verified by tests
Different rules through one interface	Achieved through pure virtual functions
Safe dynamic memory	Achieved for the normal and the duplicate-ID paths
Exception handling	Achieved; no test caused abnormal termination
Modular and extensible	Largely achieved; see the analysis below

8. Critical Analysis

A case study is only useful if it is honest about weaknesses. The report already lists several limitations. The points below confirm them and add observations that come from reading the code closely.

8.1 Strengths

The polymorphism is clean. The `Library` class contains no member-type conditionals. Adding a `ResearchScholar` class with its own numbers would require writing one new class and one menu option, with no change to `issueBook` or `returnBook`. This is the open-closed principle in action: open for extension, closed for modification.

Validation precedes mutation. Every operation checks all preconditions before changing state, so errors never leave the data half-updated.

The exception path is memory-safe. Deleting the member before throwing on a duplicate ID shows awareness that exceptions interact with ownership.

Responsibilities are well separated. `Book` guards its own state, `Member` and its children define policy, `Library` coordinates, and `main()` handles interaction and error display.

8.2 Weaknesses and risks

1. Raw-pointer ownership and copying. `Library` owns `Member*` pointers and deletes them in its destructor, but it defines no copy constructor or copy assignment operator. If a `Library` object were ever

copied, both copies would hold the same pointers and both destructors would delete them, causing a double delete. The program avoids this only because it never copies the library. This is the classic Rule of Three problem. The robust fix is to store `vector<unique_ptr<Member>>`, which makes ownership explicit, removes the manual destructor and makes accidental copying a compile-time error.

2. Unvalidated numeric input. `returnBook` accepts any integer for days kept. A negative value produces a negative `late` and therefore no fine, which is harmless, but it is still meaningless data. Likewise, nothing stops a zero or negative ID from being used.

3. Non-numeric input ends the session. The report notes that the input stream is not reset on bad input. In practice, when a letter is typed where a number is expected, `cin` enters a failed state. In C++11 and later the failed extraction stores 0 in the variable, so the menu choice reads as 0 and the loop exits. The program therefore does not hang, but it quits unexpectedly instead of showing a message. Checking `cin.fail()`, calling `cin.clear()` and discarding the bad line with `cin.ignore(...)` would make this robust.

4. Linear lookups. `findBook` and `findMember` scan the vectors, which is O(n). That is perfectly adequate at this scale but would degrade for a real catalogue. An `unordered_map<int, Book>` keyed by ID would give average O(1) lookup.

5. Pointer invalidation hazard. `findBook` returns a raw pointer into a `vector<Book>`. A later `push_back` can reallocate the vector and invalidate any such pointer. The current code does not keep pointers across calls, so it is safe today, but the pattern is fragile if the code grows.

6. State is lost on exit. All data lives in memory. This is the single largest gap between the project and a usable system.

7. Fine is a double for a currency amount. Fines here are whole rupee multiples, so there is no visible error, but a `double` is a poor choice for money in general. An integer number of paise would avoid rounding concerns if fractional rates were introduced.

8. Weak return-side integrity. `returnBook` assumes the member found from `issuedTo` is non-null. That holds in the current design because members are never removed, but it would become a null dereference the day a "delete member" feature is added without also handling borrowed books.

8.3 Design trade-off revisited

Because Student and Faculty differ only in four constant values today, the polymorphic design is, strictly speaking, more machinery than the current requirements need. Its value is forward-looking: it localises future behaviour changes. A reasonable engineer could defend either approach. What matters is that the choice is made consciously and that the report states why. In an academic setting where demonstrating OOP is itself the goal, the polymorphic design is clearly the better pedagogical choice.

9. Lessons Learned

1. **Model the problem before writing code.** Extracting classes from the nouns of the problem statement produced a design that needed almost no rework.
2. **Put variation in derived classes and shared logic in the base.** `canIssue()` is written once, yet behaves differently per member type.
3. **Protect invariants with encapsulation.** Tying `available` and `issuedTo` together behind two functions eliminates a whole category of inconsistent states.
4. **Validate everything first, then change state.** This makes operations all-or-nothing.
5. **Exceptions and ownership must be designed together.** The duplicate-member path shows that cleanup responsibility does not disappear when an error is thrown.

6 **Tests should target boundaries.** The "exactly 14 days" case is more informative than a dozen mid-range cases.

7 **Document limitations.** Recording the known weaknesses turns them into a roadmap instead of hidden defects.

10. Recommendations

The recommendations below are ordered by value against effort.

Priority	Recommendation	Benefit
High	Add input validation (`cin.fail()`, `cin.clear()`, `cin.ignore`) and reject non-positive IDs and negative days	Removes the unexpected exit and meaningless data
High	Persist data with `ifstream`/`ofstream` (for example CSV files for books and members), then consider SQLite	Makes the system usable across sessions
High	Replace `Member*` with `unique_ptr<Member>`	Eliminates manual `delete` and the double-delete risk
Medium	Store issue and due dates with `<chrono>` and compute late days automatically	Removes the manual days-kept entry and a source of operator error
Medium	Add test cases for Faculty fines and for the on-time Faculty return	Covers the second subclass in testing
Medium	Store the list of borrowed book IDs in each member	Allows a "books held by member" report and safe member deletion
Low	Switch lookups to `unordered_map`	Scales to a larger catalogue
Low	Add reservation queues, reports of defaulters and a login layer	Moves toward a complete library product
Low	Represent money as integer paise	Avoids floating-point concerns in future

A practical sequence would be: harden input, add file persistence, move to smart pointers, then introduce real dates. Each step can be built and tested on its own without redesigning the class hierarchy, which is itself evidence that the original structure was sound.

11. Conclusion

The Library Management System demonstrates how a familiar real-world process can be turned into a clean object model. By identifying five classes, placing shared behaviour in an abstract `Member` base and member-specific rules in `Student` and `Faculty`, the project achieved a `Library` class free of type checks, a `Book` class that protects its own consistency, and an error-handling scheme that keeps the program running through every invalid operation. All 18 test cases passed.

The project's limitations (in-memory storage, manual day entry, raw-pointer ownership and fragile input handling) are typical of a first version and are well understood. None of them require a change to the core design to fix, which is the strongest sign that the architecture is sound. The main takeaway for a learner is that OOP's value is not in using its vocabulary but in making change cheap: adding a new member type, a new rule or a new storage mechanism should touch as little existing code as possible. By that measure, this mini project succeeds.

12. Discussion Questions

These questions can be used for classroom discussion or self-study.

1. If a `Guest` member could borrow only one book for 7 days with a Rs. 5 fine, which files and functions would need to change? Which would not?
2. Is making `Member` abstract necessary, or would a concrete class with configurable fields be enough? Under what future requirement would the abstract design clearly win?
3. Why must `~Member()` be virtual? Describe what could go wrong if it were not.
4. Rewrite `Library` to use `vector<unique_ptr<Member>>`. Which lines become simpler and which interfaces change?
5. Design three additional test cases that target Faculty members and the boundary of the 30-day loan period.
6. How would you extend the design so that a member's list of currently borrowed books is available, and what new problems does this introduce when a book is returned?

13. References

- E. Balagurusamy, *Object-Oriented Programming with C++*, McGraw Hill Education.
- Bjarne Stroustrup, *The C++ Programming Language*, Addison-Wesley.
- Herbert Schildt, *C++: The Complete Reference*, McGraw Hill.
- Robert Lafore, *Object-Oriented Programming in C++*, Sams Publishing.
- cppreference.com, C++ reference documentation, https://en.cppreference.com.
- Source project: Hardeep Singh Sodhi, *Library Management System: OOPs Mini Project Report*, Chandigarh University, 25BCD-2(A).